

06장 임계구역과 동기화

학습목표

- 임계구역에서 발생하는 동기화 문제를 이해하고, 그 해결 방식을 단계적으로 학습한다.
- 공유 자원의 문제를 해결하기 위한 조건과, 이를 충족하기 어려운 이유를 살펴본다.
- 운영체제가 제공하는 대표적인 동기화 도구인 뮤텍스, 세마포어, 모니터의 작동 방식과 특징을 알아본다.
- 공유 자원 접근 시 발생하는 경쟁 상태를 코드로 확인하고, 주요 동기화 도구의 작동 방식을 비교·이해한다.

목차

1. 임계구역과 동기화
2. 동기화 도구
3. [심화] 동기화 코드 사례

01

임계구역과 동기화

01 임계구역과 동기화

1. 임계구역의 필요성

◆ 공유 자원 접근 시 발생하는 경쟁 상태 문제

- 둘 이상의 스레드가 동시에 공유 자원에 접근하면 예상하지 못한 문제가 발생.
- 누가, 언제 데이터를 읽고 쓰느냐에 따라 실행 결과가 달라질 수 있기 때문.

01 임계구역과 동기화

1. 임계구역의 필요성

◆ 공유 자원 접근 시 발생하는 경쟁 상태 문제



그림 6-1 공유 자원과 경쟁 상태

01 임계구역과 동기화

1. 임계구역의 필요성

◆ 공유 자원 접근 시 발생하는 경쟁 상태 문제

■ 경쟁 상태

- 둘 이상의 스레드가 동일한 공유 자원에 동시에 접근하여 실행 결과가 예측할 수 없게 되는 상황

■ 임계구역

- 둘 이상의 스레드가 동시에 접근해서는 안 되는 공유 영역

01 임계구역과 동기화

1. 임계구역의 필요성

◆ 공유 자원 접근 시 발생하는 경쟁 상태 문제

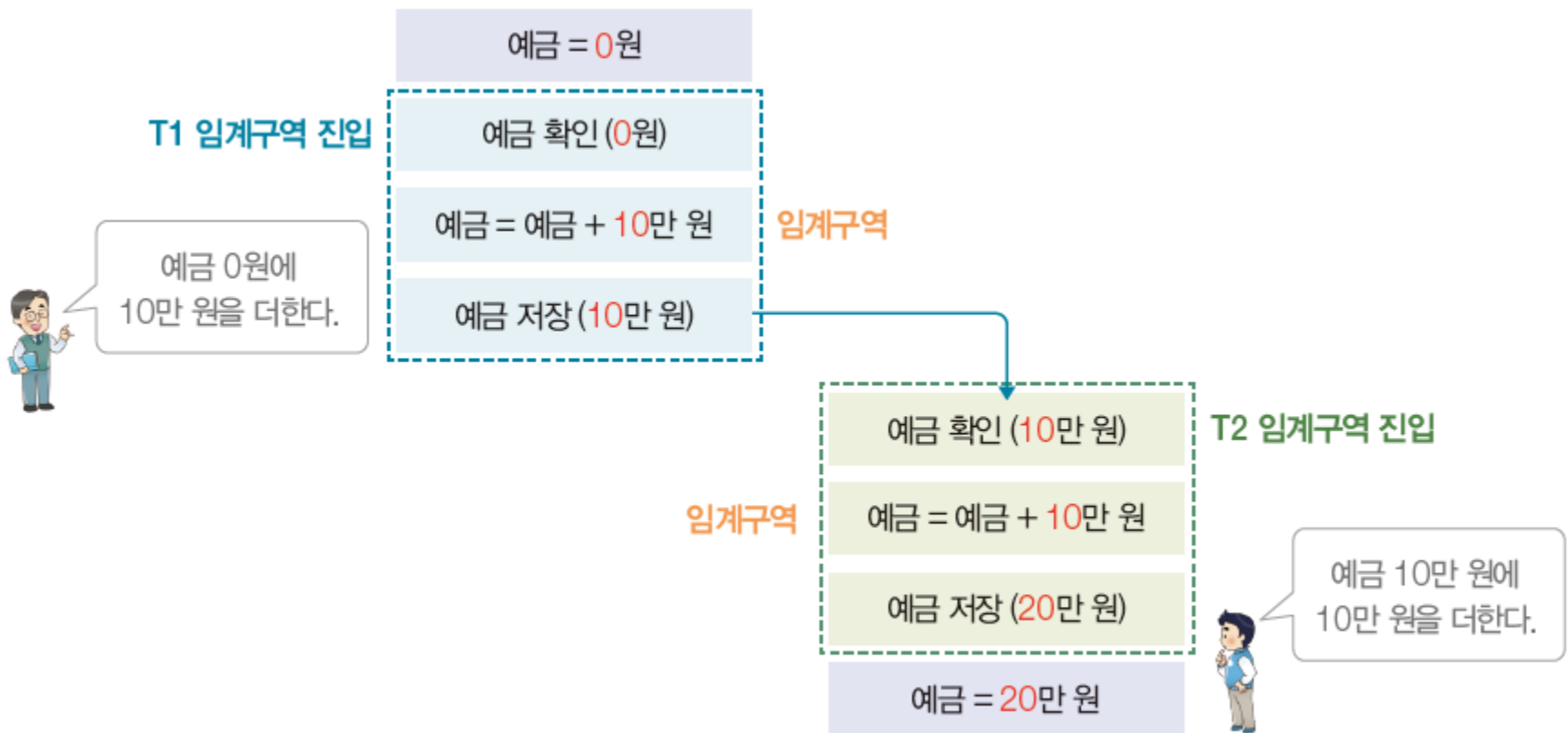


그림 6-2 임계구역과 동기화

01 임계구역과 동기화

1. 임계구역의 필요성

◆생산자-소비자 문제 사례

- 계속해서 데이터를 생산하여 버퍼에 저장하는 생산자와 버퍼에서 데이터를 꺼내 사용하는 소비자 사이에서 발생하는 문제

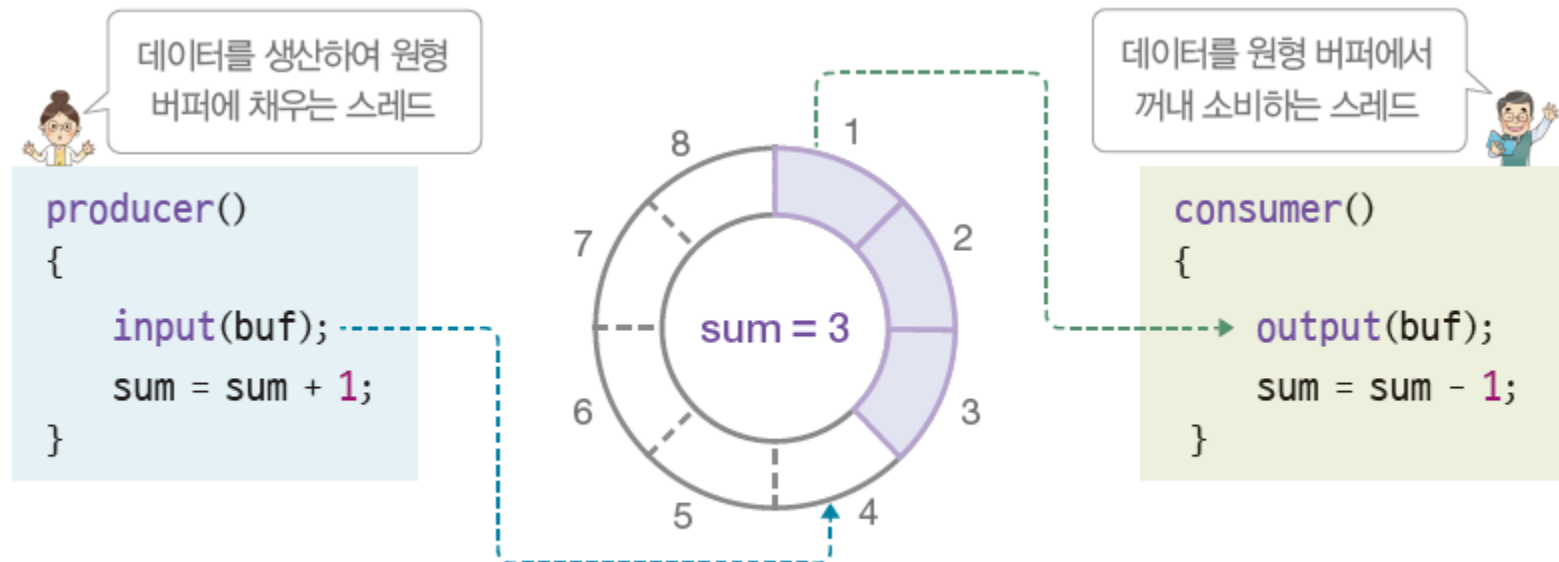


그림 6-3 생산자-소비자 문제

01 임계구역과 동기화

1. 임계구역의 필요성

◆ 생산자-소비자 문제 사례

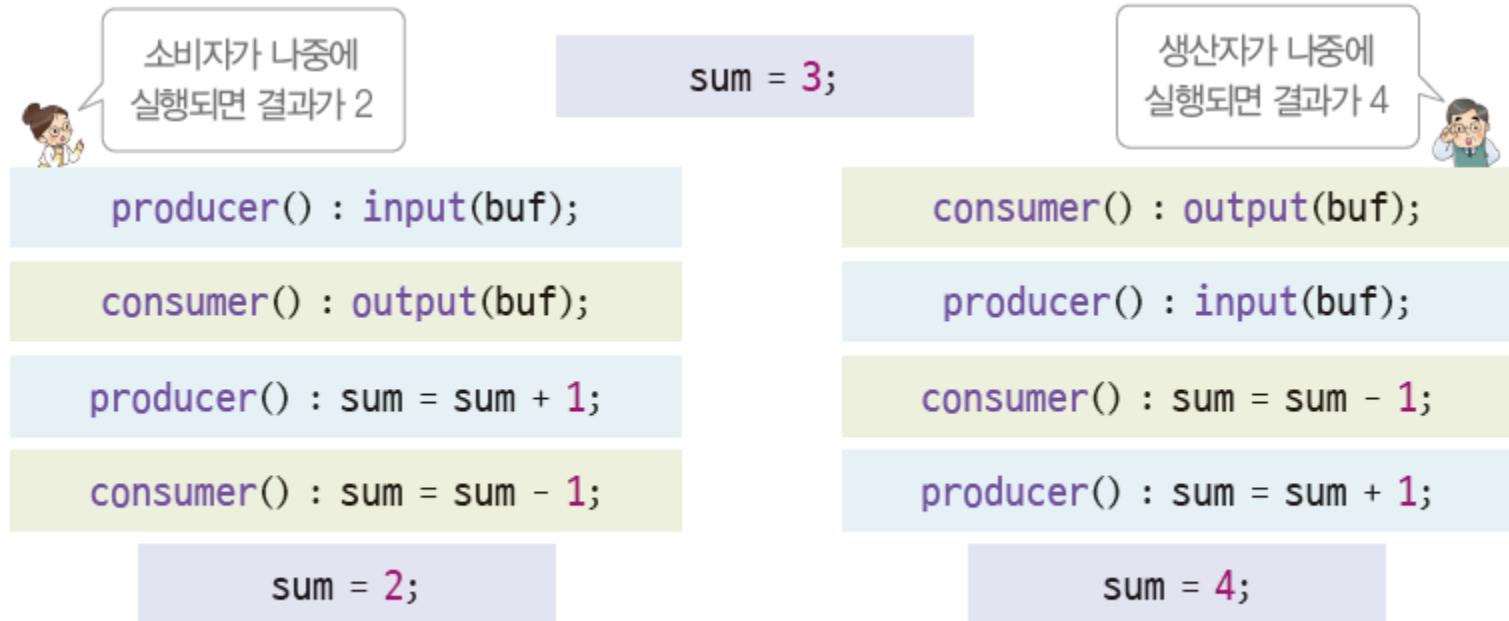


그림 6-4 실행 순서에 따른 결과 차이

01 임계구역과 동기화

2. 임계구역 문제의 해결 조건

◆ 임계구역 잠금과 해제를 통한 보호 방식

- 스레드는 잠금을 걸고 난 후, 임계구역에 진입하고, 임계구역을 나올 때는 잠금 해제와 동시에 동기화.

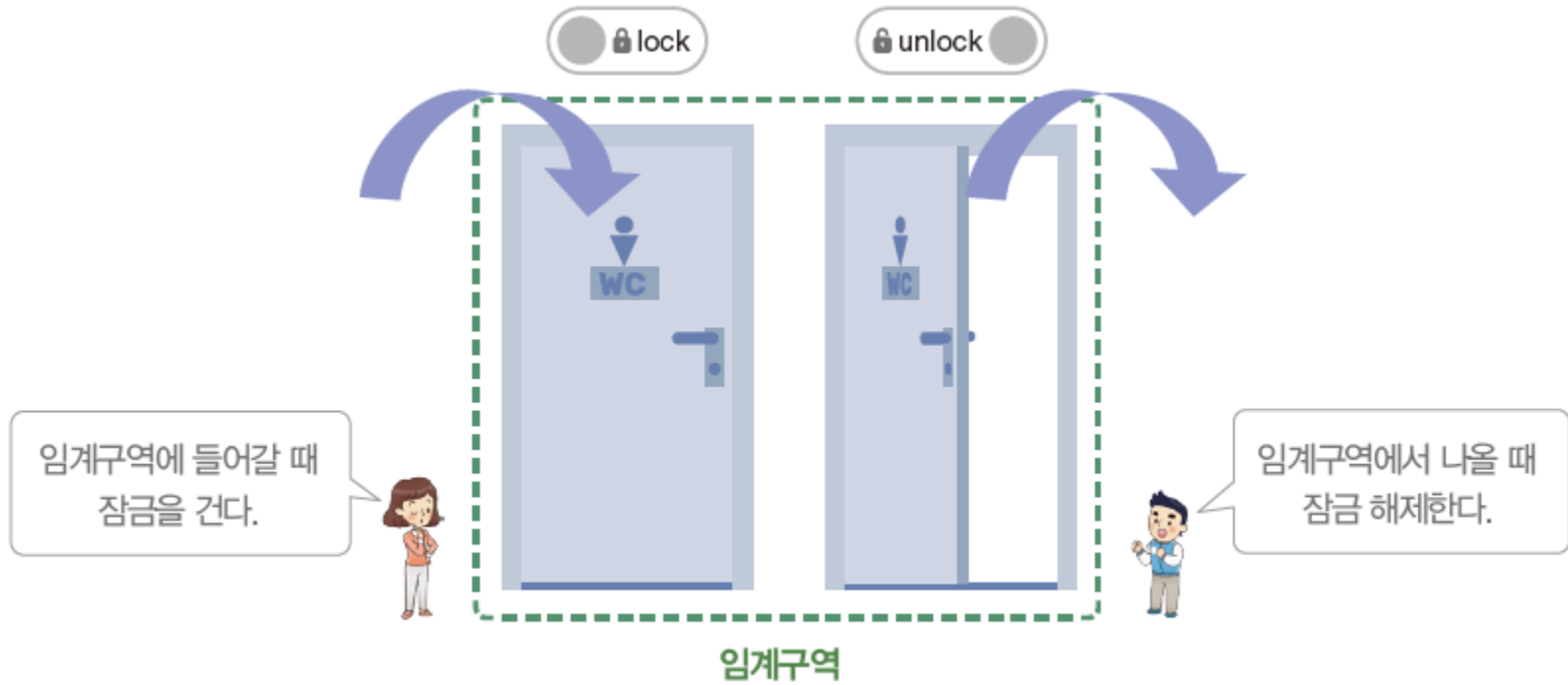


그림 6-5 임계구역 잠금과 잠금 해제

01 임계구역과 동기화

2. 임계구역 문제의 해결 조건

◆임계구역 잠금과 해제를 통한 보호 방식

- 상호 배제: 한 스레드가 임계구역에 들어가면 다른 스레드는 들어갈 수 없음.
- 한정 대기: 임계구역에 진입하려 무한히 기다리는 스레드가 있어서는 안 됨.
- 진행의 융통성: 한 스레드의 작업이 임계구역에 진입하려는 다른 스레드의 진행을 방해해서는 안 됨.

01 임계구역과 동기화

2. 임계구역 문제의 해결 조건

◆ 상호 배제 조건 위반 사례

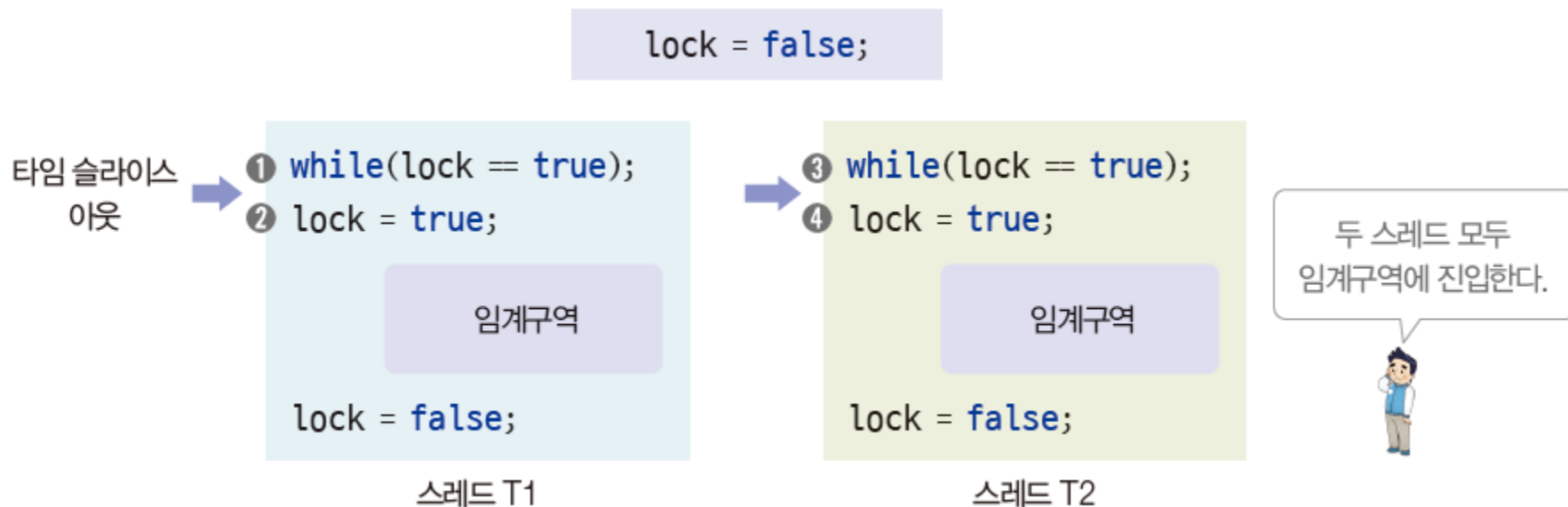


그림 6-6 상호 배제 조건 위반

01 임계구역과 동기화

2. 임계구역 문제의 해결 조건

◆ 한정 대기 조건 위반 사례

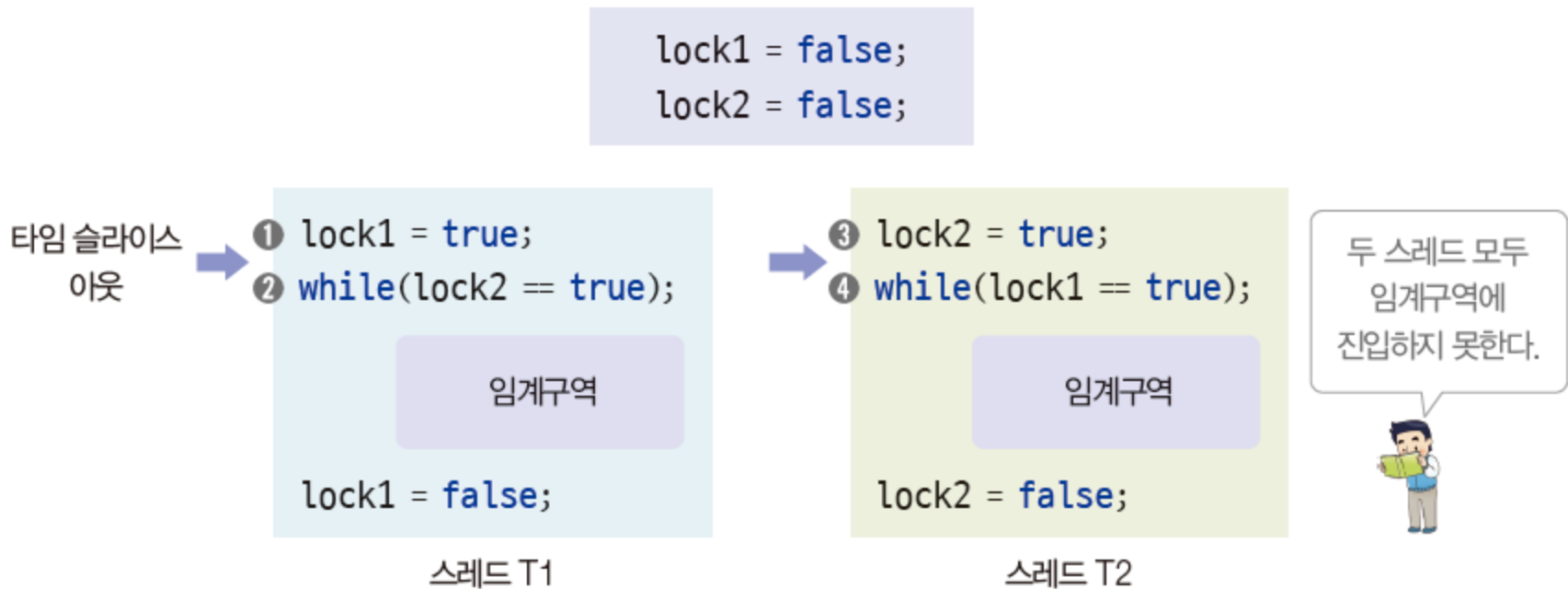


그림 6-7 한정 대기 조건 위반

01 임계구역과 동기화

2. 임계구역 문제의 해결 조건

◆진행의 융통성 조건 위반 사례

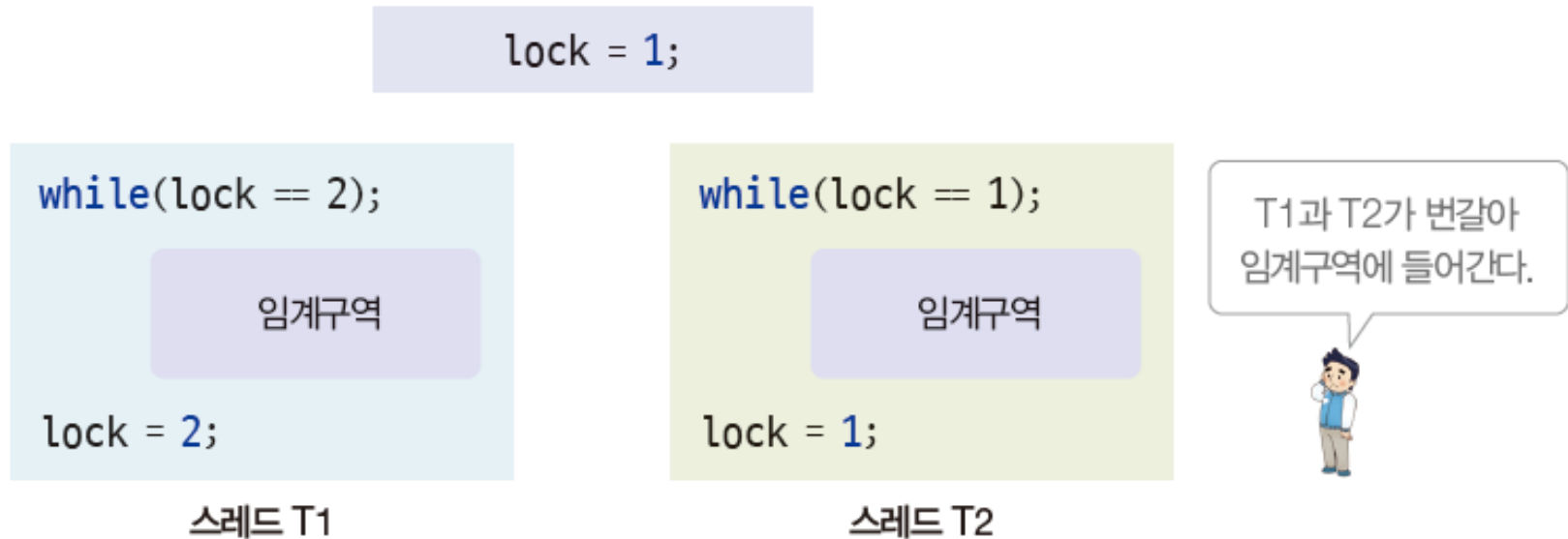


그림 6-8 진행의 융통성 조건 위반

01 임계구역과 동기화

3. 임계구역 문제의 하드웨어적 해결 방식

■ 검사와 지정

- 하드웨어 명령으로 상호 배제, 한정 대기, 진행의 융통성 조건을 위반하는 문제들을 해결
- 두 개의 연산을 원자적으로 묶음

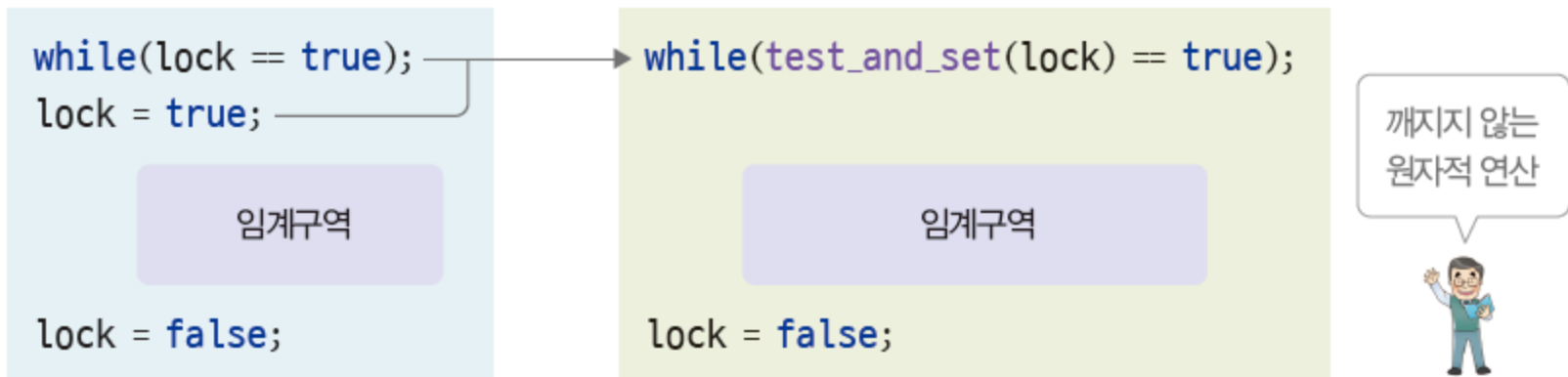


그림 6-9 검사와 지정 코드

02

동기화 도구

02 동기화 도구

1. 뮤텝스

◆ 뮤텝스

- MUTual EXclusion(상호 배제)
- 임계구역을 보호하기 위해 사용하는 기본적인 동기화 도구

lock()으로 잠금을 걸고
임계구역에 진입한다.



```
mutex_lock();
```

임계구역

```
mutex_unlock();
```

unlock()으로
잠금을 해제한다.

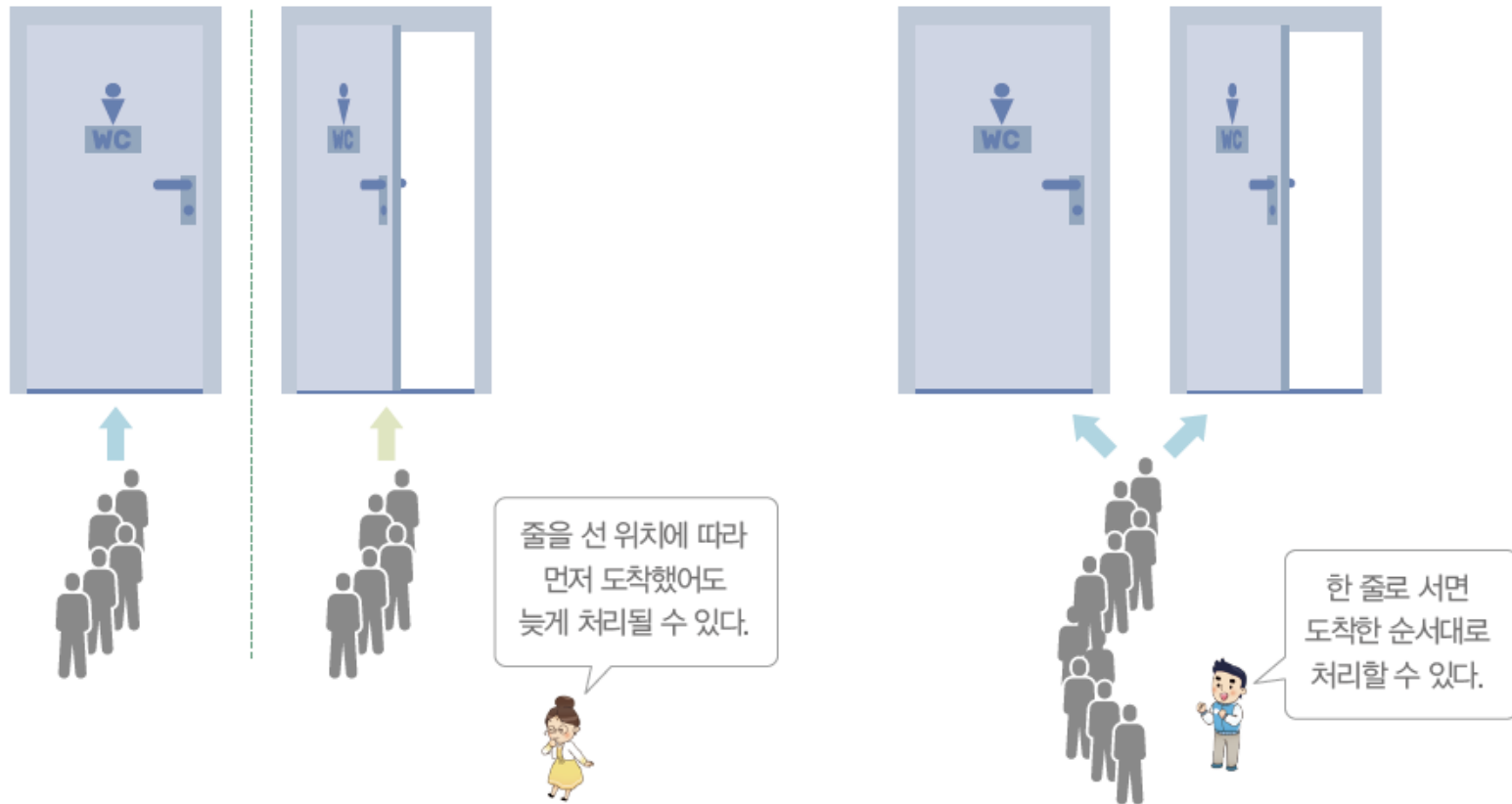


그림 6-10 뮤텝스 사용법

02 동기화 도구

2. 세마포어

◆세마포어의 개념과 구조



(a) 뮉텍스

(b) 세마포어

그림 6-11 자원이 여러 개일 때의 임계구역 처리

02 동기화 도구

2. 세마포어

◆ 세마포어의 개념과 구조

- 자원을 사용하려는 스레드가 큐에 순서대로 대기하도록 함

`init(n)`은 자원의 개수
*n*으로 초기화한다.



```
init(n);
```

```
wait();
```

임계구역

```
post();
```

`wait()`로 잠금을 걸고
`post()`로 잠금을 해제한다.



그림 6-12 세마포어 사용법

02 동기화 도구

2. 세마포어

◆ 세마포어의 개념과 구조

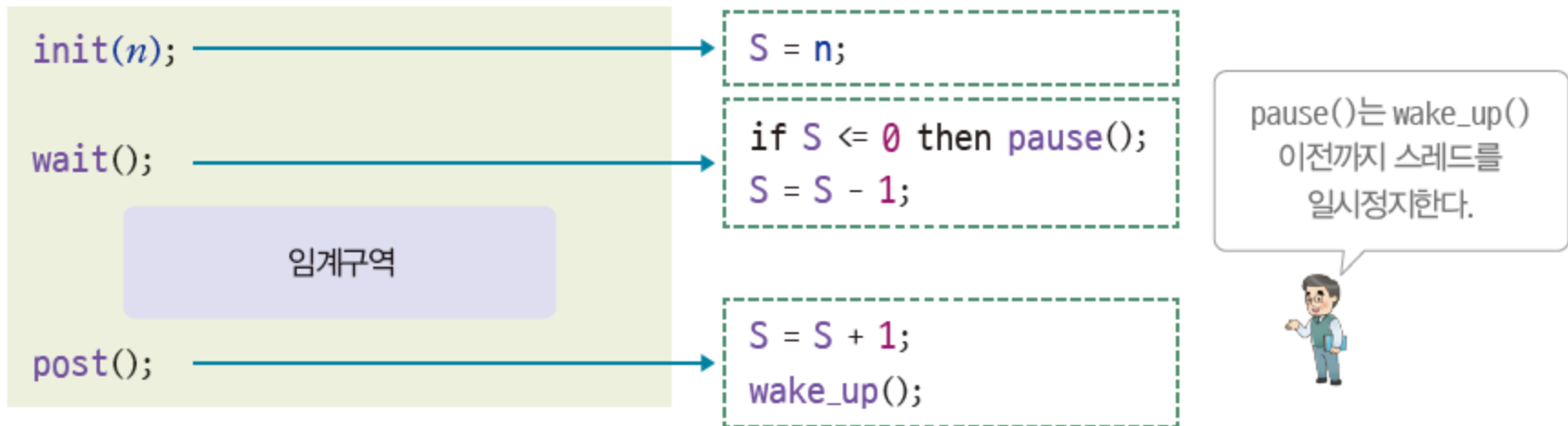


그림 6-13 세마포어 내부 코드

2. 세마포어

- 세마포어와 뮤텍스의 차이
 - 뮤텍스와 달리 세마포어는 여러 개의 자원을 동시에 관리 가능
 - 스레드의 진입 순서 제어 가능
 - `init(n)`는 세마포어를 초기화하는 함수이며, `n`은 사용 가능한 자원의 개수를 나타냄.

02 동기화 도구

2. 세마포어

◆ 세마포어 작동 원리

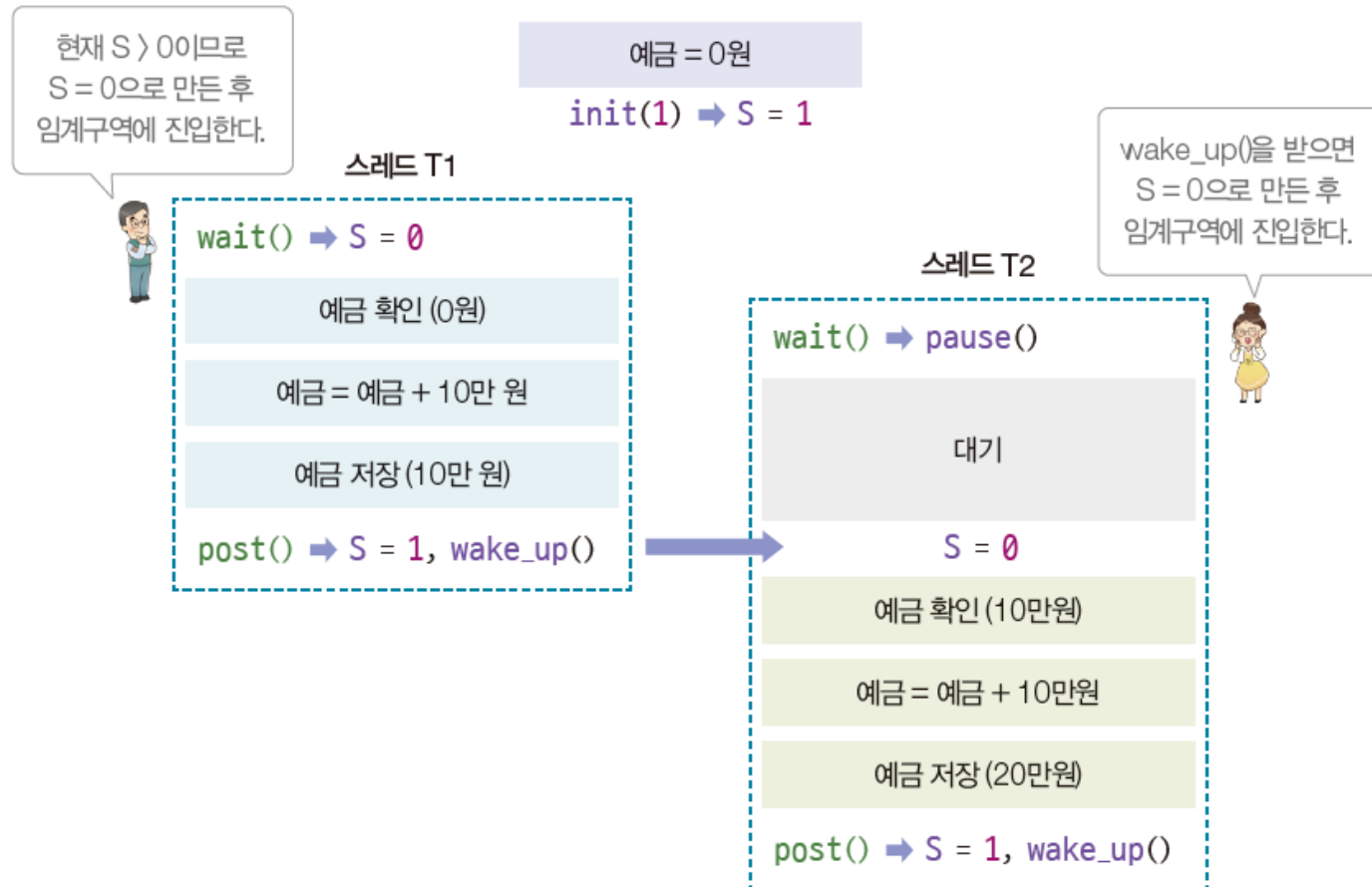


그림 6-14 단일 자원에서의 세마포어 사용

02 동기화 도구

2. 세마포어

◆ 세마포어 작동 원리

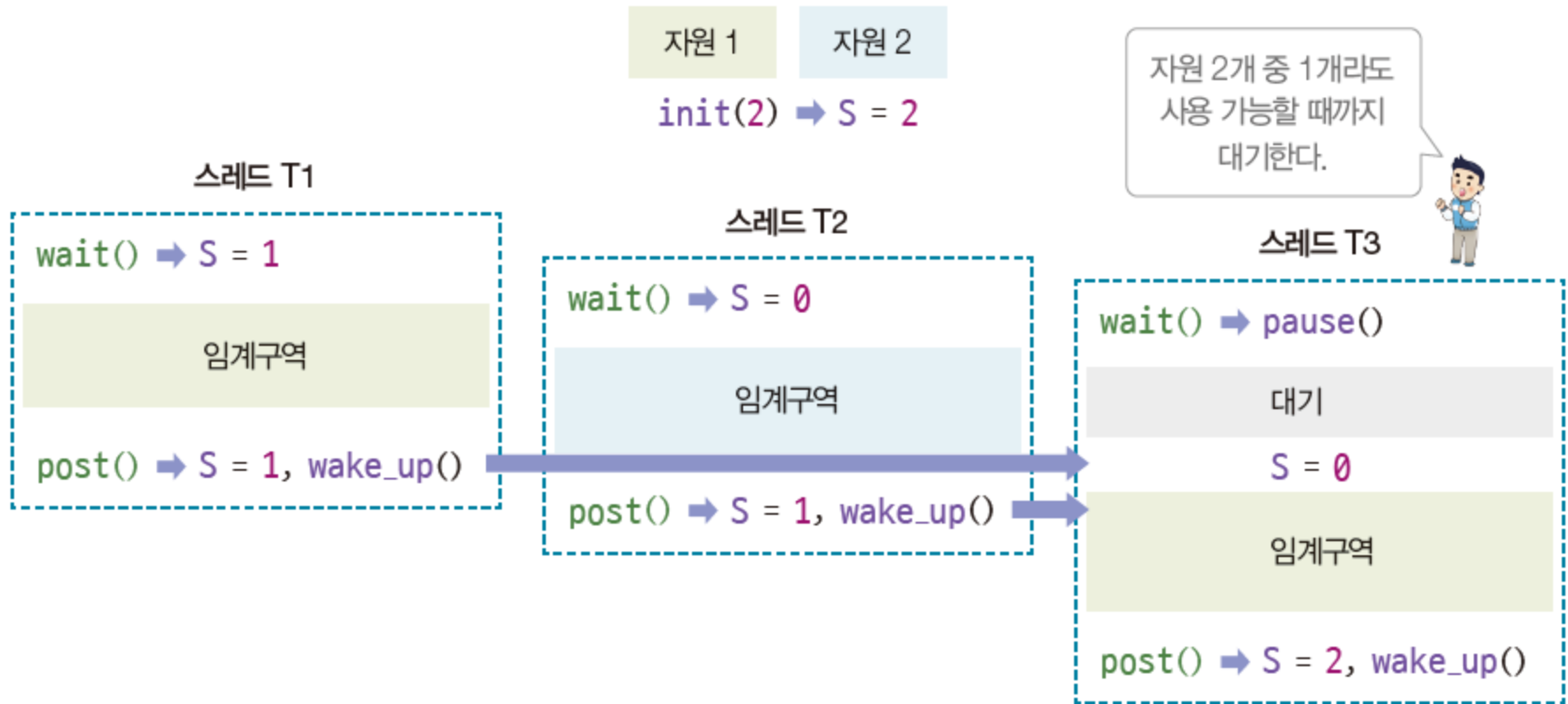


그림 6-15 자원이 여러 개일 경우의 세마포어 사용

02 동기화 도구

3. 모니터



세마포어를 잘못 사용하면
임계구역을 보호할 수 없다.

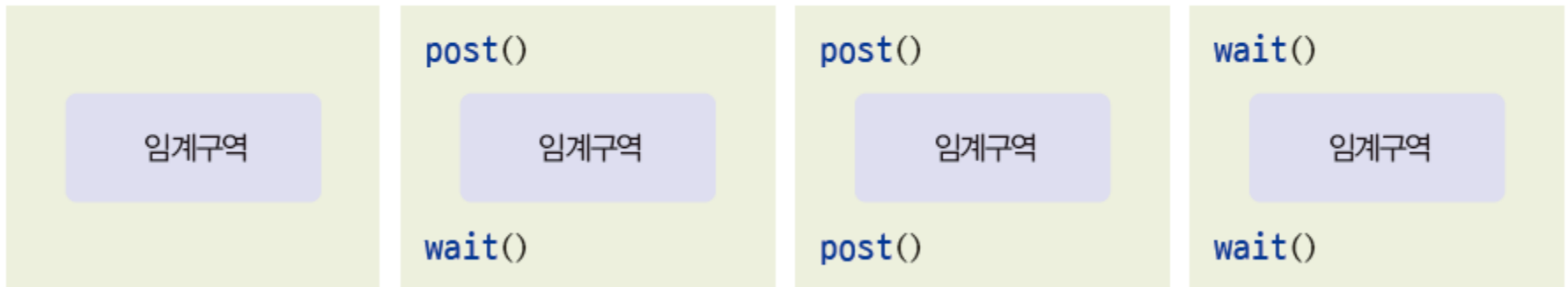


그림 6-16 세마포어를 잘못 사용하는 사례

- 모니터: 공유 자원을 안전하게 보호하고 임계구역에 대한 동기화를 자동으로 처리하는 고수준 동기화 도구

02 동기화 도구

3. 모니터

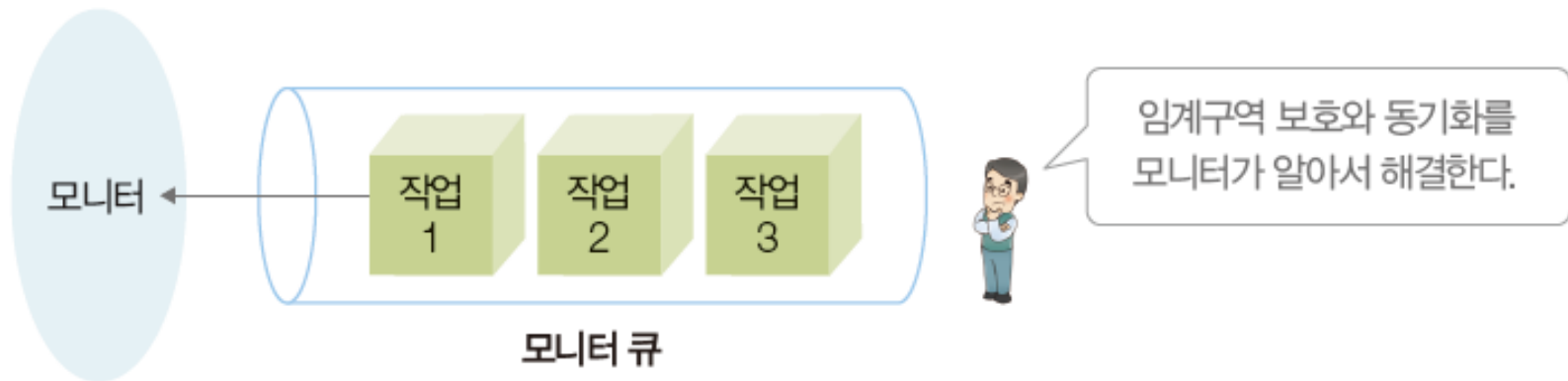


그림 6-17 모니터의 임계구역 보호

02 동기화 도구

3. 모니터

코드 6-1 자바로 구현한 모니터 코드

```
01 class MonitorCode {
02     private int dep = 0;
03     private Object lock = new Object();
04
05     public void increase(int amt) {
06         synchronized(lock) { // 임계구역 시작
07             dep = dep + amt;
08             System.out.println(dep);
09         } // 임계구역 끝
10     }
11 }
```

02 동기화 도구

3. 모니터

스레드 T1

```
increase(10);
```

스레드 T2

```
increase(10);
```

그림 6-18 모니터 코드 사용법

02 동기화 도구

4. 동기화에서 발생하는 문제: 우선순위 역전

- 우선순위 역전: 임계구역에 진입하거나 공유 자원에 접근할 때 우선순위가 낮은 프로세스 때문에 우선순위가 높은 프로세스가 지연되는 현상.

02 동기화 도구

4. 동기화에서 발생하는 문제: 우선순위 역전



그림 6-19 우선순위 역전

02 동기화 도구

4. 동기화에서 발생하는 문제: 우선순위 역전

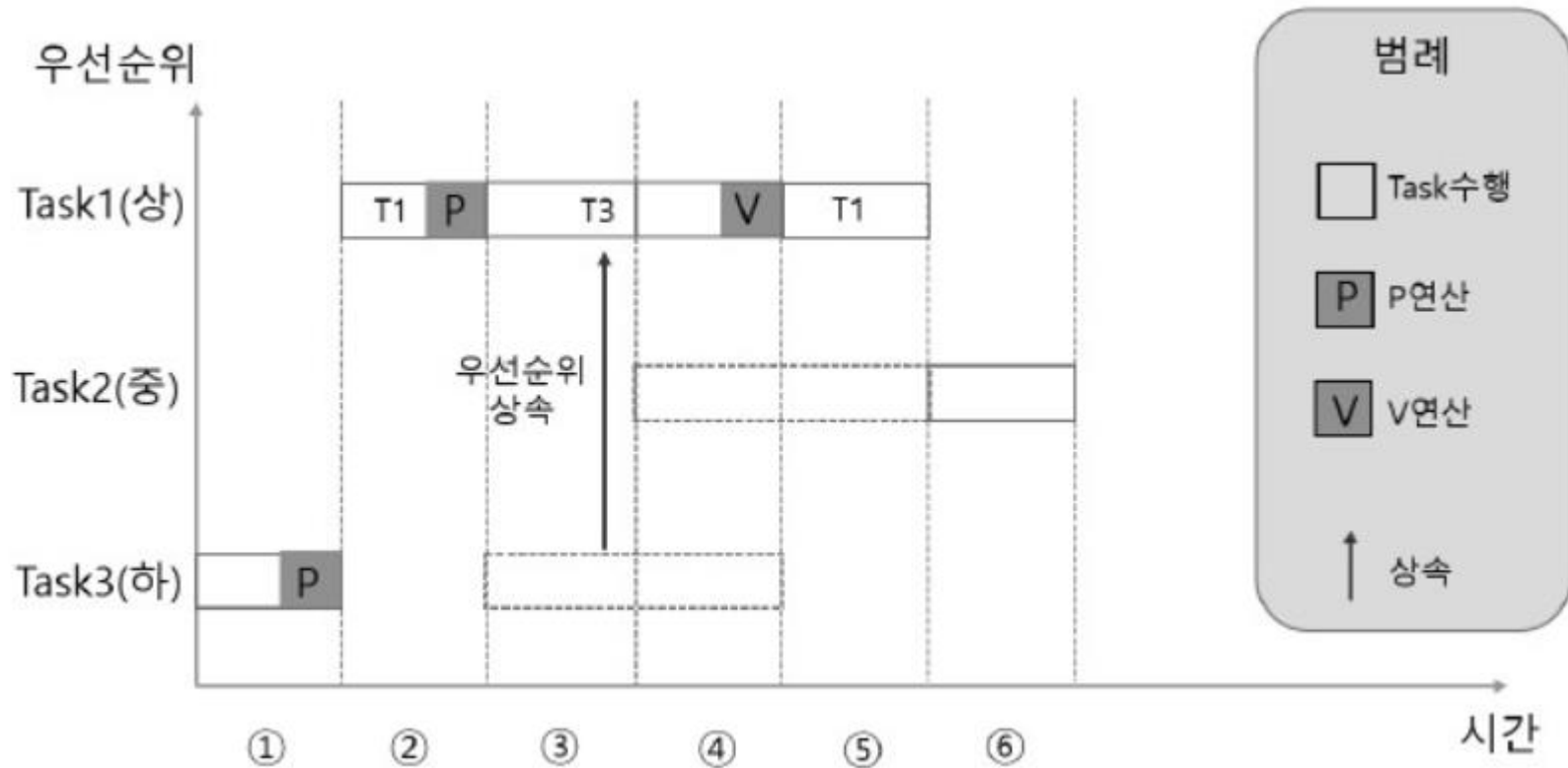
■ 우선순위 역전 해결법

- 우선순위 상속: 높은 우선 순위가 상속되면 중간에 다른 스레드가 끼어들어 작업이 지연되는 것을 막을 수 있음.
- 우선순위 상한: 임계구역을 사용하는 모든 스레드의 우선순위를 일정 수준까지 높임

구분	우선순위 상속	우선순위 올림
적용 시점	충돌이 발생했을 때	자원 획득 시점부터
방식	기다리는 고우선순위의 우선순위를 잠시 상속	미리 정한 상한 우선순위로 올림
장점	동적이고 필요한 경우만 적용	예측 가능성 높음, 역전 사전 방지
단점	관리 복잡, 오버헤드 가능	상한 설정이 어렵고 과도하면 병행성 저하

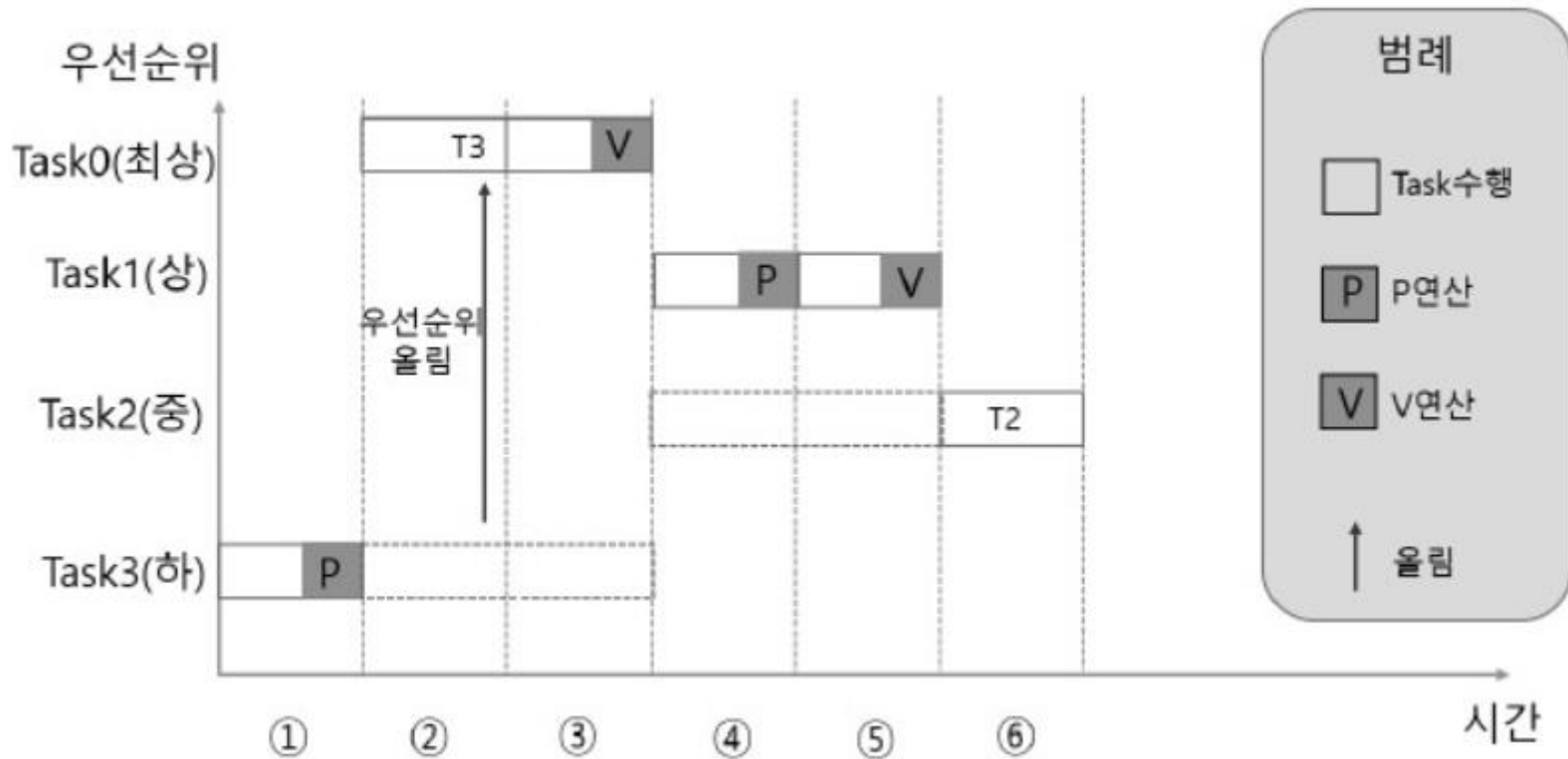
02 동기화 도구

■ 우선순위 상속



02 동기화 도구

■ 우선순위 올림



03

[심화] 동기화 코드 사례

03 [심화]동기화 코드 사례

1. 경쟁 상태 발생 코드 분석

코드 6-2 경쟁 상태 코드

```
01 #include <stdio.h>
02 #include <unistd.h>
03 #include <pthread.h>           // sleep()이 있는 라이브러리
04
05 int dep = 0;                   // 예금 초깃값 0
06
07 void *ThreadTest(void *name) { // 스레드 본체
08     char *t_name = (char *)name; // 스레드 매개변수 문자열로 변환
09
10     for (int i = 0; i < 3; i++) {
11         dep = dep + 10;           // 10만 원씩 3번 입금
12         printf("Td %s: dep %d\n", t_name, dep);
13         sleep(1);
14     }
15     pthread_exit(NULL);          // 스레드 종료
16 }
17
18 int main() {
19     pthread_t tid1, tid2;         // 스레드 구조체
20
21     pthread_create(&tid1, NULL, ThreadTest, "1"); // 스레드1 실행
22     pthread_create(&tid2, NULL, ThreadTest, "2"); // 스레드2 실행
23     pthread_join(tid1, NULL);
24     pthread_join(tid2, NULL);
25 }
```

03 [심화]동기화 코드 사례

1. 경쟁 상태 발생 코드 분석

```
Td 1: dep 10
Td 2: dep 20
Td 1: dep 30
Td 2: dep 30    <- 여러 번 실행하면 가끔씩 10이 사라짐
Td 1: dep 40
Td 2: dep 50    <- 결과가 60이어야 하는데 50
```

03 [심화]동기화 코드 사례

2. 뮤텁스를 이용한 임계구역 보호

코드 6-3 뮤텁스 코드

```
01  #include <stdio.h>
02  #include <pthread.h>
03  #include <unistd.h>
04
05  int dep = 0;                // 예금 초깃값 0
06  pthread_mutex_t mlock;     // 뮤텁스 구조체 선언
07
08  void *ThreadTest(void *name) { // 스레드 본체
09      char* t_name = (char *)name; // 스레드 매개변수 문자열로 변환
10
11      pthread_mutex_lock(&mlock); // 잠금 획득(lock)
12      for (int i = 0; i < 3; i++) {
13          dep = dep + 10;          // 10만 원씩 3번 입금
14          printf("Td %s: dep %d\n", t_name, dep);
15      }
16      pthread_mutex_unlock(&mlock); // 잠금 해제(unlock)
17      pthread_exit(NULL);
18  }
19
```

03 [심화]동기화 코드 사례

2. 뮤텁스를 이용한 임계구역 보호

```
20 int main() {  
21     pthread_t tid1, tid2;  
22  
23     pthread_mutex_init(&mlock, NULL); // 뮤텁스 초기화  
24     pthread_create(&tid1, NULL, ThreadTest, "1");  
25     pthread_create(&tid2, NULL, ThreadTest, "2");  
26     pthread_join(tid1, NULL);  
27     pthread_join(tid2, NULL);  
28 }
```

```
Td 1: dep 10  
Td 1: dep 20  
Td 1: dep 30  
Td 2: dep 40  
Td 2: dep 50  
Td 2: dep 60    <- 스레드가 순서대로 실행된다
```

03 [심화]동기화 코드 사례

3. 세마포어를 이용한 동기화 구현

코드 6-4 세마포어 코드

```
01 #include <stdio.h>
02 #include <pthread.h>
03 #include <unistd.h>
04 #include <semaphore.h>           // 세마포어가 있는 라이브러리
05
06 int dep = 0;                     // 예금 전역변수, 초기값 0원
07 sem_t smp;                       // 세마포어 구조체 선언
08
09 void *ThreadTest(void *name) {   // 스레드 본체
10     char* t_name = (char *)name;
11
12     sem_wait(&smp);               // 세마포어 wait() - lock
13     for (int i = 0; i < 3; i++) {
14         dep = dep + 10;           // 10만 원씩 3번 입금
15         printf("Td %s: dep %d\n", t_name, dep);
16     }
17     sem_post(&smp);               // 세마포어 post() - unlock
18     pthread_exit(NULL);
19 }
```

03 [심화]동기화 코드 사례

3. 세마포어를 이용한 동기화 구현

```
20
21 int main() {
22     pthread_t tid1, tid2;
23
24     sem_init(&smp, 0, 1); // 세마포어 init(1) - 공유 자원 1
25     pthread_create(&tid1, NULL, ThreadTest, "1");
26     pthread_create(&tid2, NULL, ThreadTest, "2");
27     pthread_join(tid1, NULL);
28     pthread_join(tid2, NULL);
29 }
```

```
Td 1: dep 10
Td 1: dep 20
Td 1: dep 30
Td 2: dep 40
Td 2: dep 50
Td 2: dep 60    <- 스레드가 순서대로 실행된다
```


03 [심화]동기화 코드 사례

4. 자바 모니터를 이용한 동기화 구현

코드 6-5 자바 모니터 코드

```
01 class MonitorCode {
02     private int dep = 0;
03     private Object lock = new Object();
04
05     public void increase(int amt, int no) {
06         synchronized(lock) {           // 모니터 - 스레드 동기화
07             for (int i = 0; i < 3; i++) {
08                 dep = dep + amt;
09                 System.out.println("Td " + no + ": " + dep);
10             }
11         }
12     }
13 }
14
15 class ThreadTest extends Thread {
16     MonitorCode mt;
17     int no;
```

03 [심화]동기화 코드 사례

4. 자바 모니터를 이용한 동기화 구현

```
18
19     ThreadTest(MonitorCode mt, int no) {
20         this.mt = mt;                // 매개변수 mt를 ThreadTest의 mt로 변환
21         this.no = no;                // 매개변수 no를 ThreadTest의 no로 변환
22     }
23     public void run() {
24         mt.increase(10, no);         // MonitorCode의 increase() 호출
25     }
26 }
27
28 public class Main {
29     public static void main(String args[]) {
30         MonitorCode mo = new MonitorCode();
31         ThreadTest t1 = new ThreadTest(mo, 1);
32         ThreadTest t2 = new ThreadTest(mo, 2);
33         t1.start();
34         t2.start();
35     }
36 }
```

03 [심화]동기화 코드 사례

4. 자바 모니터를 이용한 동기화 구현



```
Td 1: 10  
Td 1: 20  
Td 1: 30  
Td 2: 40  
Td 2: 50  
Td 2: 60
```

Thank You!